

ARCHITECTURE SELECTION

We chose Microservices Architecture for the Airport Lost Luggage Portal.

Architectural Choice

The platform is decomposed into independently deployable services for claims, tracking, bag matching, notifications, and compensation, fronted by an authenticated API Gateway. Each service owns a cohesive business capability and communicates through explicit APIs or status events. Baggage scan data and claim state remain behind dedicated data interfaces.

Reason 1 - Independent scaling across transit hubs

Baggage tracking and matching process high-volume scan streams from many airport terminals, while claim entry and compensation have different load profiles. Microservices allow the Tracking and Bag Matching services to scale independently during peak travel without duplicating the entire platform. New airports can be connected through the scan repository interface.

Reason 2 - Fault isolation and workflow evolution

A temporary notification or compensation failure must not stop passengers from registering claims or agents from viewing scan timelines. Separate services contain failures and can retry status events. Airline compensation policies and bag-matching logic can also evolve and deploy independently without disrupting the core claim workflow.

Security Advantage

The API Gateway provides one controlled entry point for authentication, authorization, rate limiting, and TLS. Service identities and least-privilege policies restrict scan, passenger, and compensation data to the components that require them. Compensation operations can use stronger agent roles and audit logging without exposing that capability to passenger sessions.

Performance Benefit

The Tracking Service can cache recent multi-leg scan timelines and use read-optimized indexes, while the Bag Matching Service processes scan feeds asynchronously. This prevents expensive correlation work from delaying passenger requests and supports the required sub-200 ms timeline response under simulated peak load.